AGI ALPHA FLYWHEEL

From Completed Work to Compounding Capability

Strategy as a Productive Asset



MAKE BETTER WAYS TO WORK.

SUCCESSOR Ω HARNESS

ACTIVATE. EVOLVE. SCALE.

Powered by TRANSFORMATIONAL INTELLIGENCE ∞
and the applicable $AGIALPHA utility.

Vincent Boucher

Montréal, Québec, Canada / 24 October 2026

A distinct edition. Preferred Edition 02 remains unchanged.

RESEARCH & ENGINEERING EDITION / EXECUTED EVIDENCE

Strategy as a Productive Asset

A bounded experiment in discovering, retaining and building on better methods

Abstract. Can completed work improve the method of doing the next job? We implement a bounded strategy-succession experiment through the unchanged released AGI ALPHA workbench: worker-constrained release planning, typed dispatch/lookahead/repair methods, separate correctness and investment gates, current result checks, persistent history and explicit stopping. We first recover and audit the original Strategy Laboratory, then freshly replay its six frozen-policy attempts and 27,648 confirmation cases in the recorded runtime. Those benchmark results are not inherited economic proof.

The new source and protocol are frozen before one launch executes 336 stream/treatment trials and 72 native-reference calls. At the primary cumulative budget, the flywheel qualifies 517/864 offered missions versus 465 for a saved workflow (paired mean difference 4.33; 95% bootstrap interval [3.67, 4.92]). The equally enabled conventional learner matches it exactly, and the stronger fixed portfolio qualifies 591. No new asset is retained in any primary arm: the apparent advantage over the saved workflow cannot be attributed to inheritance. At the larger cap, all arms qualify 668/864 and the flywheel costs 16.10% more than the portfolio.

A separately budgeted diagnostic constructs 12 typed method lineages. Fresh dispatch-to-lookahead execution and repair qualification reuse are verified, but no fresh repair both invokes its parent and qualifies. Inherited ancestry uses 60.35% less work than exact reconstruction, and equivalent replacements preserve every fresh schedule. Yet removing ancestry and using the conventional fallback qualifies 37/48 missions versus 36/48 for inheritance at much lower cost. We therefore establish bounded method composition and conditional acquisition reuse, not a qualified three-generation production chain, net delivery advantage, self-acceleration or recovered full research investment. A complete offline browser planner, frozen protocols, raw failures, independent implementation audit, editable figures and reproducible source accompany the paper.

The result in one sentence

A mission can leave a method used by a real successor. Whether that successor earns its cost requires a separate test.

Question	Evidence disposition
Does the bounded recurring loop run?	Yes: one launch, automatic permitted phases, persistent results and tested recovery.
Does a method help construct a distinct child?	Yes in construction/qualification and fresh dispatch-to-lookahead work; a qualified fresh third-stage use is absent.
Does retained strategy beat strong delivery alternatives?	Not established. Primary acquisition fails; the strong portfolio wins, and diagnostic removal is cheaper.
Does the enterprise self-accelerate or repay all research?	Not established. No endogenous reinvestment, paid adoption or complete monetary labor accounting.

Use it now. Open the accompanying `AGI_ALPHA_Strategy_Laboratory.html` and select *Plan this mission*. The default computes and checks a new schedule. Recorded research is labeled separately. No account, wallet or API key is required.

1 The mission should leave a better way to work

A useful project plan answers a present question: which work should start when, with the available crew? A reusable strategy answers a different question: how should the next plan be constructed, checked and repaired? The second object can be productive capital. It also has an acquisition cost, an applicability boundary and a maintenance liability. Keeping every discovered method is therefore an investment policy, not a free improvement.

This edition studies that distinction through the released AGI ALPHA workbench. Its flagship is a finite release-planning service: give it jobs, dependencies, durations, crew needs, a capacity calendar and a deadline; receive a schedule with a separately checked feasibility record. A later requirement can change the calendar and lock already committed work. The system must then repair the plan, account for disruption and decide whether a retained method is still useful. Every example uses disclosed synthetic inputs. A computed schedule is not evidence that the corresponding physical work was performed.

The contribution is an executed, inspectable *strategy-succession experiment*: bounded method search inside the existing workbench, separate correctness and investment gates, fresh mission evaluation, constructive ancestry interventions, recovery records and a browser companion. The strategy grammar contains familiar specialist methods. The scientific claim concerns their construction, selection, reuse and cost in a recurring system; no new general scheduling algorithm or uniquely agentic advantage is assumed.

The decisive question

Can the enterprise discover a better way to work, establish when it helps, retain it, and use it to improve a subsequent worthwhile mission—after paying for the attempt?

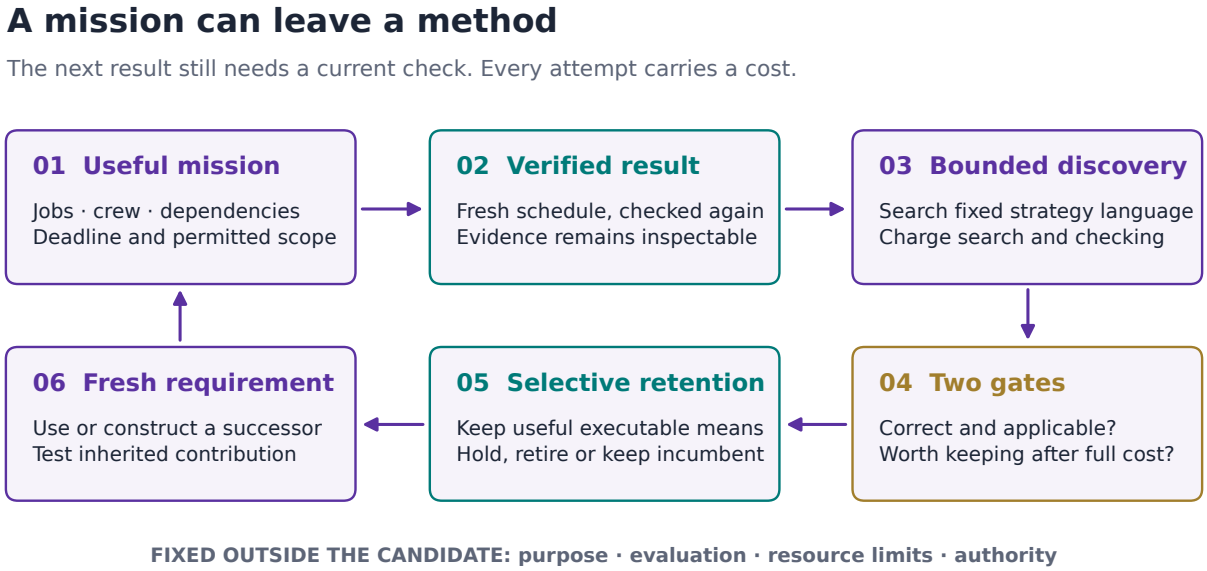


Figure 1: A productive loop with two gates. Discovery, rejection and upkeep are charged; current purpose, evaluation and authority stay outside candidate control.

1.1 What would count as growth?

Let $Q_i \in \{0,1\}$ indicate that a fresh mission satisfies its frozen acceptance contract. Let C_i include all declared attributable computational work up to that result, including unsuccessful construction. For cumulative limit B, define qualified coverage

$$K(B) = \sum_i Q_i \, \mathbf{1}\{C_i \le B\}. \tag{1}$$

Coverage is assessed on an offered stream, so an early stop does not remove later missions from the denominator. A rejected or over-budget mission is an observed outcome. Throughput and elapsed time are separately measured; abstract operation charges are never converted silently into seconds or money.

Useful reuse means a retained executable method contributes to a newly checked result. **Constructive succession** additionally requires that the inherited method helps construct or qualify a semantically different capability. **Amortized productivity growth** requires a full-cost advantage against a matched strong alternative. **Capability expansion** requires more qualified fresh missions under the same budget. **Increasing growth rate** requires improving marginal productivity across matched resource windows, beyond merely repaying an initial cost. **Commercial growth** requires evidence of demand and economic outcomes. These are separate hypotheses.

The minimum operational claim is narrower: after one explicit start, the finite selector can complete successive permitted phases, preserve history, hold an incumbent, refuse a candidate, wait, recover and stop. Successful execution of that loop does not imply any of the stronger growth claims.

1.2 Transformational Intelligence as a testable discipline

The foundational AGI ALPHA paper places governed jobs, verification and reusable capabilities above a particular model [1]. Transformational Intelligence separates the mission purpose, its formulation and the means that realize it [2]. Here the purpose is fixed: produce a valid plan within stated constraints. The system can change the strategy that constructs the plan, while current acceptance criteria remain outside the strategy's control.

This experiment deliberately gives conventional controls the same strategy language. A useful discovered expression therefore establishes strategy learning within that language; it does not establish escape from its attainable set. The possibility of changing a restriction is a research hypothesis that requires a purpose-preserving bridge and its own causal evidence. The symbol ∞ expresses the research programme, not an unbounded resource grant or an achieved intelligence level.

Operation	Executed research role	Inspectable output
Identify	Select the next permitted mission from the frozen queue	Mission, scope, remaining budget and stop/defer decision
Out-Learn	Evaluate bounded candidate priorities on disclosed development tasks	Candidate evaluations, failed attempts, acquisition debit
Out-Think	Challenge a proposed schedule against original evidence in the separate development probe	Reproducible objection or rejected criticism
Out-Design	Construct a typed dispatch, lookahead or repair strategy	Content-bound executable AST and dependency record
Out-Strategise	Compare current qualified-output forecasts after full charges	Separate admission decision and break-even conditions
Out-Execute	Invoke the unchanged workbench adapter and check its current result	Schedule, check, receipt and cost ledger
Out-Evolve	Retain, replace, retire or hold; use permitted means in a later construction	Successor lineage, fresh qualification and removal tests

Table 1: Operational counterparts of the six Out-stage papers and Identify. The challenge/adjudication probe is separately executed; it is not an inline model debate in every automatic mission. This mapping does not claim to instantiate every theoretical result in the source papers.

Out-Learn motivates investing in an experiment; Out-Think motivates checked representations and their retirement; Out-Design emphasizes executable release certificates; Out-Strategise couples information gathering to construction; Out-Execute addresses reusable boundaries and local repair; Out-Evolve makes the durable output a means of further discovery and production [3–8]. The present study makes those responsibilities concrete without inheriting their conclusions.

2 Recovering the Strategy Laboratory before extending it

The supplied `202610180921.docx` was recovered with the original *SUCCESSOR Ω—Strategy Laboratory v0.1.0*. Its 62 internal payload hashes match. Source, protocols, development populations, frozen candidates, confirmation inputs, summaries, history and replay logs survive. We audit these originals and replay the frozen policies [9, 10].

A separately written current audit regenerated all 27,648 committed confirmation inputs, matched every task hash, rechecked 193,536 stored policy/control artifacts and recomputed six decisions. The original integrity verifier and 51 tests also pass in the current Python 3.12.14 environment. We then obtained the recorded CPython 3.13.5 runtime and freshly executed the original verifier with `--replay`. All six attempts and 27,648 cases pass exact equality of non-timing policy/control outputs, judgments and decisions. The replay took 78.344 seconds on the current host; no source edit or runtime-guard bypass was used. The record and log are preserved under `history/current-exact-replay/`.

These are two complementary checks: an independently written artifact/statistical audit and fresh execution of the original frozen-policy implementation. The latter is not independent institutional replication and does not rerun development search as a new historical experiment. Historical timing comparisons remain the original measurements; elapsed time is intentionally excluded from exact replay equality.

Attempt	Domain	Incumbent	Candidate	Nominal gain LB	Decision
1	Route	.082543	.824725	+.712524	Promote
2	Evidence	.877693	.891657	−.001014	Hold
3	Schedule	.660756	.675751	−.003020	Hold
4	Route	.829620	.917588	+.062721	Promote
5	Evidence	.874611	.890156	−.002091	Hold
6	Schedule	.660477	.698699	+.018489	Promote

Table 2: Recomputed historical decisions. Each attempt uses 3,072 nominal and 1,536 shifted tasks. Scores are the historical bounded task objectives, not full-cost value. LB is the original empirical-Bernstein lower-bound formula, independently recalculated.

2.1 What survives the stronger comparison

The historical search considered 432 evolutionary and 432 random candidates with equal candidate-evaluation counts. It correctly held both evidence-selection nominees. The first route promotion nevertheless scores .824725 against .890632 for the fixed weight-2 A* reference. The second route nominee scores .917588 against .892152 for that specialist and .849101 for the random-search nominee. These last two comparisons are descriptive: the registered promotion hypothesis compares each candidate with its incumbent, not with every alternative.

On the bounded historical domain, the final arithmetic priority simplifies to minimizing

$$g + 1.398870138\,h + 0.997674183\,\max(2.203, \text{terrain}). \tag{2}$$

It is an interpretable terrain-augmented weighted-A* priority. The fixed strong reference did not receive an equally enabled conventional weight-and-terrain tuning search. Equal exploration counts also did not imply equal total work: the final candidate used more expansions, priority evaluations and expression nodes. Its recorded nominal-plus-shift solver interval was 1.616 times the strong reference's interval. Those timers excluded task generation, the Dijkstra oracle, judging, serialization and history work. A score improvement therefore supplies no historical full-cost break-even result.

Repeated testing was accounted for inside the preserved pilot/final pair. Six pilot attempts spend approximately .04285714 of the error allowance; the final workspace reserves .00714286, with its first six attempts spending .00612245. The combined observed spending is approximately .04897959. This does not license arbitrary new workspaces, unregistered strong-control superiority tests or post-hoc benchmark selection. The historical shifted gate also tolerates a lower bound down to −.08; it is not a no-regression guarantee. Full details, timings, seed intersections and controls appear in the runnable historical audit.

2.2 The research lead: make disagreement earn its place

The DOCX proposes a useful chain: creator, evidence-grounded challenger, independent adjudicator, revision and regression test. The challenger must see the original evidence; a criticism is itself a claim to check. In this edition a challenge identifies a concrete schedule violation, such as overlapping crew demand or a moved locked job. The adjudicator recomputes the disputed predicate. A substantiated counterexample becomes a development regression asset; an unsupported criticism is refused. Protected confirmation cases are never promoted into training during the experiment.

This is executable deterministic checking, not a live multi-model debate. The independently implemented checker shares the same commission and host as the constructor. Its implementation separation reduces one route to correlated error, but does not establish institutional independence or a cryptographic proof of the entire research process. Candidate access is restricted by the AST language; a privileged host administrator remains outside that threat model.

2.3 Continuity without inherited conclusions

The Possibility Foundry already demonstrates a simulated instrument helping create a distinct temporal-rate capability, while reporting cheaper conventional construction and an unfavorable ancestor-inclusive affordability result [11]. The earlier Harness detector likewise ties a cached conventional controller and loses cost to direct expert construction [12]. Those findings motivate the present strong controls. They are not rerun evidence for this scheduling study.

The neural-symbolic SUCCESSOR Ω constitution separates WORLD, POLICY, PROOF and AUTHORITY; Mission Gym calls for versioned evaluation and credible alternatives; SUCCESSOR Ω× AGI JOBS supplies bounded work/proof contracts and authority non-composition [13–15]. Preferred published Edition 02 remains unchanged at its existing URL and verified hash [16]. This distinct edition adds strategy search, selective admission and a changed planning/repair contract; it does not overwrite an earlier experiment's scope or result.

3 A capability has a contract, a lineage and a carrying cost

For a task t, let $P_a(t)$ execute method a, and let $V(t, y)$ be the fixed independent checker for result y. An asset contains the executable strategy, its input/output contract, source and dependency digests, development observations, qualified scope and lifecycle history. It contains no transferable current verdict or production grant.

3.1 Two gates, two different questions

The correctness/applicability gate checks that the task lies in the declared scope and that its output satisfies current constraints. A valid schedule may still miss the requested deadline or exceed the allowed repair movement. Qualification includes those user-facing requirements. A method's own favorable report cannot substitute for the independent result check.

The investment gate asks whether retaining the exact candidate is expected to improve qualified output under the remaining mission count and resource budget. If $\hat{q}_a$ is its qualification rate on the fixed qualification set, $\hat{c}_a$ its full measured marginal cost, A_a its acquisition debit, M_a its prospective maintenance allocation, N the remaining same-scope offered missions and R the resource budget available immediately before that acquisition debit, a finite forecast is

$$\widehat{K}_a(R, N) = \hat{q}_a \min\left\{N, \left\lfloor \frac{(R - A_a - M_a)_+}{\hat{c}_a} \right\rfloor\right\}. \tag{3}$$

The implementation compares this forecast with the counterfactual incumbent that did not incur A_a; a tied output forecast needs a positive full-cost saving. The forecast is an admission policy trained on a small disclosed set. It is not an unbiased future-value estimate or a statistical guarantee. Subsequent fresh outputs receive current correctness checks; the primary experiment does not calibrate this forecast because its flywheel acquisition produces no completed nominee.

Sunk and prospective costs must be distinguished in a current decision, while both remain in the scientific lifecycle ledger. Once a search has been paid for, rejecting its output cannot erase that debit. Conversely, a useful asset should not be charged its already-paid acquisition again at every reuse. Descendant construction must pay for ancestry that was actually rebuilt. The removal and replacement interventions make those accounting choices explicit.

The registered code's `net_benefit` field is therefore a *full-investment forecast*, not an optimal marginal adoption rule after discovery. A method can be worthwhile to use now even when it cannot repay earlier research within the remaining stream. For example, an already-paid acquisition of 100 units and a prospective five uses saving 10 units each give useful marginal reuse but leave 50 units of the original investment unrecovered. This arithmetic example is not an experimental result. The conservative gate's semantics are reported as

implemented; the primary run, which never completes its flywheel nominee, does not estimate that gate's practical effect.

Verdict	Evidence required	What it does not establish
Benchmark promotion	Frozen score comparison and its registered uncertainty rule	Practical usefulness, cheaper delivery or investment payback
Practical adoption	Applicable, currently checked work whose outcome and burden justify use in its actual setting	Recovery of earlier research costs or commercial success
Research investment recovered	Cumulative realized benefit exceeds acquisition, failed research, checking, upkeep and attributable intervention	Indefinite growth or an increasing rate of improvement

Table 3: Three verdicts that cannot be substituted for one another. This study's local admission is a forecast-based research decision. Actual outside adoption and recovery of unknown authoring costs are not observed.

Proposition 1 (Finite payback condition)**.** *For equal-quality work, suppose a candidate costs A to acquire, saves s per future mission relative to the incumbent, and costs m per mission to retrieve, check and maintain beyond the incumbent's burden. Its declared cumulative saving after N missions is positive exactly when $s > m$ and $N > A/(s-m)$.*

Proof. The difference is $N(s-m)-A$. If $s \leq m$, no nonnegative mission count repays a positive acquisition debit. Otherwise rearranging the strict positivity inequality gives the stated threshold. □

This accounting identity clarifies an investment boundary; it is not a new economic theorem. For unequal qualification rates, the coverage forecast and the incremental cost per additional qualified task replace a misleading equal-output speedup. Any monetary valuation requires an externally supplied value and an observed resource price. Synthetic computational savings are not revenue.

3.2 What the successor inherits

A dispatch strategy ranks currently eligible jobs using a bounded linear expression over disclosed features. A lookahead strategy evaluates possible next decisions by running an inherited dispatch strategy to complete each provisional suffix. A repair strategy adds a changed contract: preserve locked work and trade off completion time against displacement from a baseline, with an option to use an inherited planner for the remaining suffix. A stable repair can instead preserve baseline order without invoking that planner. These contracts differ, but storing a parent in an AST does not prove that an executed branch uses it.

The constructor must record which exact parent was read and executed, which fresh construction or qualification task it served, and which child bytes resulted. A digest is necessary for attribution but insufficient for utility. Removing the parent, rebuilding it from the same available methods, and supplying an independently reconstructed equivalent parent answer different questions: dependency, acquisition saving and dependence on the useful semantics rather than the asset's name.

All arms retain access to competent specialist algorithms. Serial schedule generation and priority rules are established project-scheduling methods [17]; using a base heuristic to evaluate provisional decisions is the classical rollout idea [18]. The present implementation does not inherit every improvement guarantee of unrestricted rollout: bounded width, truncated search, repair objectives and resource cutoffs alter its conditions. A candidate can be slower, worse or inapplicable.

3.3 Bound discovery, maintenance and forgetting

The controller searches only at registered opportunities, under an explicit debit limit, using a small new training batch and a disjoint qualification batch. It does not rescan the entire growing history after every job. A denied investment leaves the competent incumbent available. A stale dependency invalidates applicability; lack of useful remaining work produces defer or wait; an exhausted budget, deadline or authority scope produces stop or refusal. The archive records unsuccessful proposals without allowing their historical verdicts to become current admission.

Source inspection, AST validation, candidate execution, current checking, retention decisions and history work all have costs. The measured computational tariff is disclosed in the frozen protocol. It is a modeling

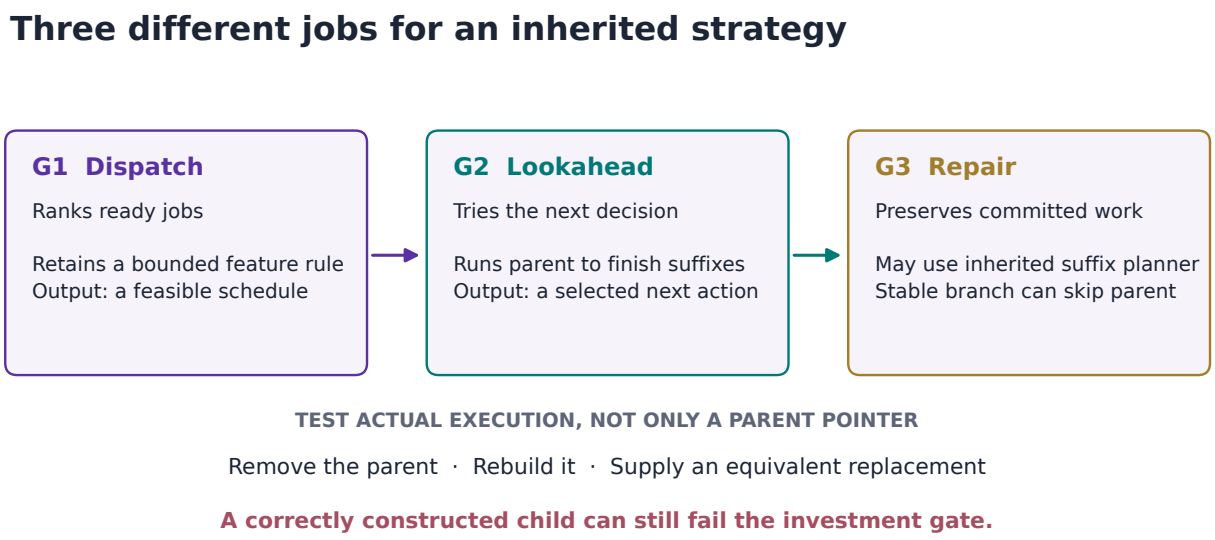


Figure 2: The intended constructive dependency changes the method's contract. The diagnostic interventions distinguish a real inherited dependency from the economic decision to retain its child. Exact executed outcomes follow in the results.

instrument rather than a count of all physical machine instructions. Wall time, CPU time, stored bytes and intervention counts provide complementary views. Unmeasured authoring effort and human attention remain unknown; they are never silently entered as zero.

4 A frozen experiment on useful release plans

4.1 Mission distribution and protected evaluation

The confirmatory design has 12 independent streams of 72 offered missions. Each stream contains three 24-mission windows with eight ordinary plans, eight tighter-deadline plans and eight repair tasks. Ordinary instances have 10 or 12 jobs; tighter and repair instances have 14. Durations are integers from 1 to 9, pooled capacity is 2, 3 or 4 workers, and precedence is generated as a directed acyclic graph under a prespecified sparse/dense mixture. The horizon is 128 discrete slots.

Deadlines are drawn from fixed slack multipliers of a registered structural lower bound, not selected from a candidate's output. Ordinary slack is 1.20 or 1.35, tight slack is 1.00 or 1.08, and repair slack is 1.08 or 1.20. A structural bound need not be attainable, so a missed target can be a hard or infeasible request rather than an invalid plan. Each repair uses a separately generated conventional baseline, not the preceding accepted mission's plan; it includes predecessor-closed locks, releases and a changed calendar. The main repair qualification requires feasibility, preserved locks and the deadline. Movement from the baseline is descriptive; optional movement limits are supported and tested separately.

The configuration for each stream is materialized once and reused byte-for-byte across methods and budgets. Discovery sees only six training tasks and four disjoint qualification tasks at each stage. Protected mission inputs are unavailable to candidate search. A global registry checks seed/scope and semantic task hashes across development, browser fixtures, independent-checker probes, load inputs, training, qualification and confirmation. It removes superficial task labels before checking equality; it does not test arbitrary graph isomorphism. The released source, strategy grammar, checker, analysis and resource protocol are frozen before execution. The unexecuted checkpoint and prospective coordination record make that order inspectable.

4.2 Strong alternatives have the same opportunity to succeed

All delivered schedules receive the separate final checker. Speculative internal portfolio attempts are search computations, not individually accepted deliveries. The conventional learner is intentionally allowed to match the flywheel exactly: strategy learning is not unique to an agent label. Hand-authored specialist rules and the native optimization model are transparent expert-built software references, not a timed trial

Arm	Frozen behavior
Portfolio	Competent fixed priority-rule portfolio, including a cheap baseline-preserving repair option; no newly acquired methods.
Saved workflow	Select, check, cache and adapt conventional specialist recipes using the same disclosed training and qualification access.
Conventional learner	Exactly the same bounded strategy-learning mechanism as the flywheel; only the outer experimental label differs.
Flywheel	Discover dispatch, construct lookahead from retained dispatch, construct repair from retained lookahead; admit only after both gates.
No discovery	Execute the strong system with newly acquired capabilities disabled.
No admission	Retain a correctly qualified method without requiring its investment forecast to win.
No retention	Charge search and checks, then discard a proposed retained method.
No succession	Construct later methods from the supplied competent base recipes rather than the acquired parent.

Table 4: The primary comparison is flywheel minus saved workflow. Portfolio and the equally enabled learner prevent a forgetful or artificially weak baseline from supporting a uniqueness claim. Ablations may coincide when a gate holds every candidate; such coincidences are reported.

of independent human experts.

A separate native reference uses SciPy 1.17.0's HiGHS mixed-integer solver [19, 20]. Its prospectively selected subset is the first ordinary, tight and repair task in each stream, 36 tasks total. Both deadline-feasibility and makespan modes receive a two-second solve limit and a ten-second process-response ceiling. The makespan mode receives a freshly computed, independently checked portfolio upper bound, whose construction is disclosed. A zero-objective deadline model can prove feasibility without proving optimal makespan. Native model building, solve, startup, checking, CPU and end-to-end elapsed time remain separate from the JavaScript reference-work tariff; equal total native and interpreted computation is not claimed.

4.3 Resource limits and two kinds of construction

The primary cumulative budget is 1,000,000 declared work units per stream, with descriptive sensitivity runs at 250,000 and 2,000,000. A delivered output has a 100,000-unit pipeline ceiling. Discovery is offered after completed missions 1, 25 and 49, with at most 500,000 units for a stage. A stage reserves four candidate/reference pairs at 50,000 units per output before spending its exploration remainder. Pair order alternates. Equal *total pipeline* allocations do not guarantee identical solver allowances: the current history and checkpoint overhead must also be paid. Both allocations are logged, and a stage-starved comparison defers admission.

The closed menus offer eight dispatch recipes, six lookahead constructions and three repair constructions. A fixed training-only successive-halving schedule first exposes every surviving method to the same next task, keeps the best half after a completed round, and finishes the last survivor on all six tasks. Eight dispatch methods therefore need at most 17 evaluations instead of 48 full-menu evaluations. An incomplete budget-stopped round cannot decide elimination. All eliminated candidates, partial searches, failures and charges remain in the record. One exact candidate identity is fixed before fresh qualification. A stream may be unable to afford all three opportunities. There is no repeated rescanning of an ever-growing corpus and no extension of demand to manufacture payback.

A separate constructive diagnostic uses 12 fresh streams, four changed missions per stream and four treatments: inherited ancestry, removed ancestry, rebuilt ancestry and an equivalent normalized replacement. Its total cap is 5,000,000 units, stage cap 1,500,000 and qualification allocation 100,000 per output. It may retain a correct method that fails the investment gate so that its descendant can be tested. This intervention is explicitly *correctness-only retention*, not an economic promotion by the selective enterprise. Recorded calls establish whether a parent actually participates in child construction or qualification; exact outputs and replacement/removal effects establish what that dependency changes.

4.4 Accounting, uncertainty and stopping rules

Charged work includes named graph, calendar, interpreter and deterministic sorting operations; separate checker operations; source/input and envelope processing; evidence and checkpoint blocks; discovery failures; admission, lookup and maintenance; and retry/recovery work. The source defines the tariff. Wall time and CPU are separately measured. Synthetic fixture construction, including the supplied pre-disruption plan, is measured once as common WORLD-input preparation outside the conditional mission-work endpoint. It is disclosed rather than silently treated as the system's free product. Human and assistant authoring/setup costs have unknown duration and price; the experiment cannot establish all-in monetary profitability.

The primary outcome is qualified coverage of the entire offered stream at the cumulative limit. An early stop leaves the unattempted tasks in the denominator. The registered uncertainty calculation is a paired percentile bootstrap over the 12 independent streams, 10,000 resamples with seed 20261024. It does not treat correlated tasks within a stream as independent observations. There is one primary contrast; additional arms, caps, diagnostic interventions, native comparisons and productivity windows are descriptive. No post-launch exclusions, result-driven retuning or additional runs until a desired win are permitted.

Each stream has a 120-second wall limit and a two-second checker-response ceiling. The controller also enforces scope, authority expiry, finite work and STOP. A candidate cannot change the evaluation distribution, checker, budget or authority. These boundaries establish a bounded local experiment, not a production security proof or a persistent commercial service.

5 Results: construction, adoption and payback separate

The separately implemented post-run audit passes all 336 trial ledgers, 20,118 raw records, 20,083 stored-output checks, 14,595 receipt rechecks and 67 available native schedules. It reproduces the registered primary bootstrap and verifies 1,632 distinct registered inputs with no normalized overlap against preserved exposed development tasks. This is code-level independent checking within the same commissioned project, with the limitations stated below. No source or protocol is changed after the freeze.

5.1 The primary advantage is not inherited capability growth

At the registered 1,000,000-unit per-stream cap, the flywheel completes **517 qualified missions out of 864 offered**, compared with **465** for the saved-workflow control. The equally enabled conventional learner also completes **517**, with exactly the same total charged work, **11,714,786 units**. The stronger fixed portfolio completes **591**. Thus the registered contrast favors the flywheel over one conventional workflow, but that advantage does not survive the strongest counted-work alternative.

The paired primary difference is **+4.33 qualified missions per stream**, with the registered 95% percentile-bootstrap interval **[3.67, 4.92]**. All 12 paired differences are positive. This is a contrast against the saved-workflow procedure under the chosen budget, not evidence that new retained methods caused the improvement. Against the fixed portfolio, the flywheel delivers 74 fewer qualified missions in total. No post-launch exclusion or retuning is used.

Primary arm	Attempted	Valid	Qualified / 864	Charged units	New assets
Strong portfolio	769	768	**591**	11,719,575	0
Saved workflow	615	613	465	11,749,635	0
Conventional learner	677	675	517	11,714,786	0
Selective flywheel	677	675	517	11,714,786	0
No discovery	769	768	**591**	11,719,668	0
No admission	677	675	517	11,714,915	0
No retention	677	675	517	11,714,915	0
No succession	619	618	469	11,744,541	0

Table 5: Twelve paired streams per arm; cumulative cap 1,000,000 units per stream. Failed or incomplete outputs remain counted as attempts; qualification additionally requires a valid schedule meeting its deadline. All streams stop under the declared budget boundary. Tiny differences between behaviorally coincident arms can arise from their differently sized ledger labels.

The causal reason matters more than the name of the winning arm: **no newly acquired asset is retained in any primary arm**. In the flywheel arm, the search budget left after the protected qualification reserve fails to produce a complete candidate. Its subsequent parent-dependent stages cannot proceed. This is an acquisition-stage failure under the registered partition, not evidence that a completed successor failed the investment gate. The saved-workflow arm does complete one primary nominee and retains the incumbent. Lower overhead than the saved-workflow search explains a comparative advantage without demonstrating retained-strategy benefit. The no-admission and no-retention primary outcomes coincide with the flywheel; those ablations cannot identify an admission or retention effect when no asset enters either mechanism.

The independent acquisition audit identifies 12 flywheel `NO_COMPLETE_CANDIDATE` outcomes at the primary cap and no payback-admission tests. Across the 96 dispatch menu slots, 58 are normal race eliminations, 26 are unfinished prefixes and 12 carry a work/budget-truncation flag. These categories must not be conflated: elimination is an intended search decision, whereas an incomplete nominee cannot enter qualification. The source's protected check reservation is conservative enough to starve this discovery schedule. That is an observed design limitation, not evidence that all strategies are inherently uneconomic.

At the larger 2,000,000-unit cap, all arms complete all 72 missions per stream and qualify the same **668/864**. No new asset is retained. Mean full charged work is 1,157,022.3 units for the portfolio, 1,343,296.6 for the flywheel and 1,556,758.6 for the saved workflow. The flywheel therefore spends **16.10% more** than the stronger portfolio at matched qualified output. Its mean window productivity declines from 4.99 to 4.30 to 3.58 qualified missions per 100,000 charged units in the primary windows; portfolio rates also decline, from 7.12 to 4.71 to 3.84. A positive difference between two declines is not acceleration. No endogenous earned-resource reinvestment is implemented.

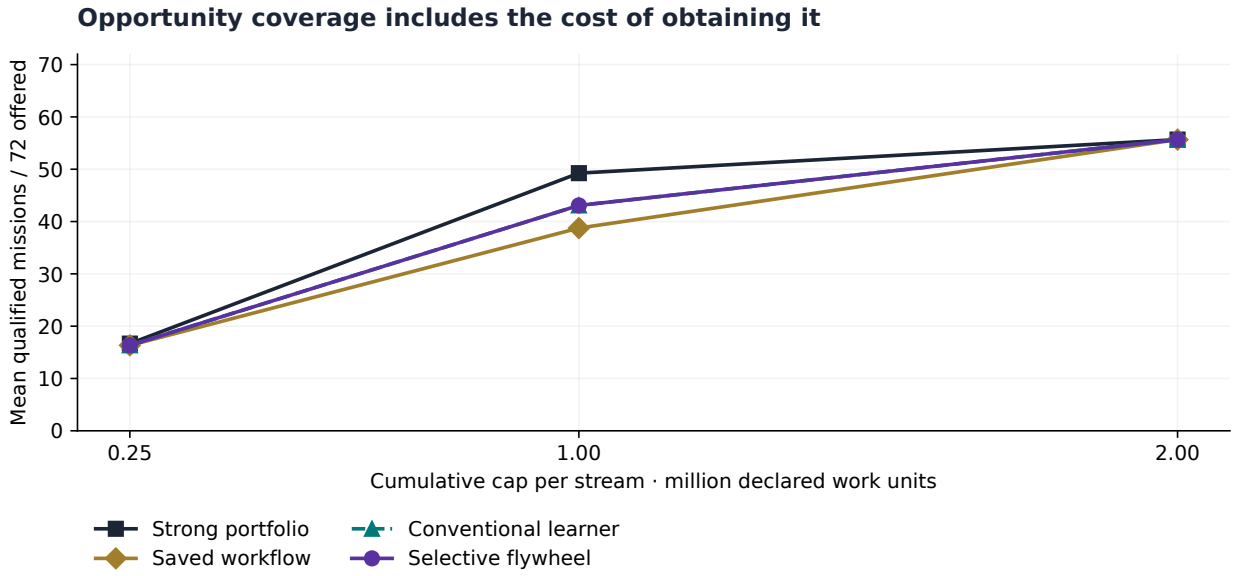


Figure 3: Qualified coverage across the three registered cumulative caps. These are paired executions on the same offered tasks, not three independent task samples. Acquisition and operating history remain inside the charged budget; stopped missions remain in the denominator.

5.2 Constructive succession under its separate research allowance

All 12 inherited diagnostic streams construct three typed, correctness-qualified assets: dispatch, lookahead and repair. This is a structural lineage count, not a count of productive three-generation executions. The selected dispatch is critical-path-first in nine streams and shortest-duration-first in three. Lookahead settings vary across six retained recipes; repair selects quality, selective or stability-oriented modes. Experience therefore determines executable policy bytes within the authored language, rather than merely renaming a fixed answer.

The diagnostic records 2,411 inherited dispatch rollouts across all research and mission phases. All 12 fresh lookahead missions actually invoke their dispatch parent, and three qualify. Repair requires a sharper distinction: its 86 aggregate parent calls comprise 23 during training of eventual winners, 44 in losing-candidate training, 18 during selected-candidate qualification and only one on fresh mission work. Four qualification outputs both invoke the parent and qualify. No fresh repair does both. Ten retained stable

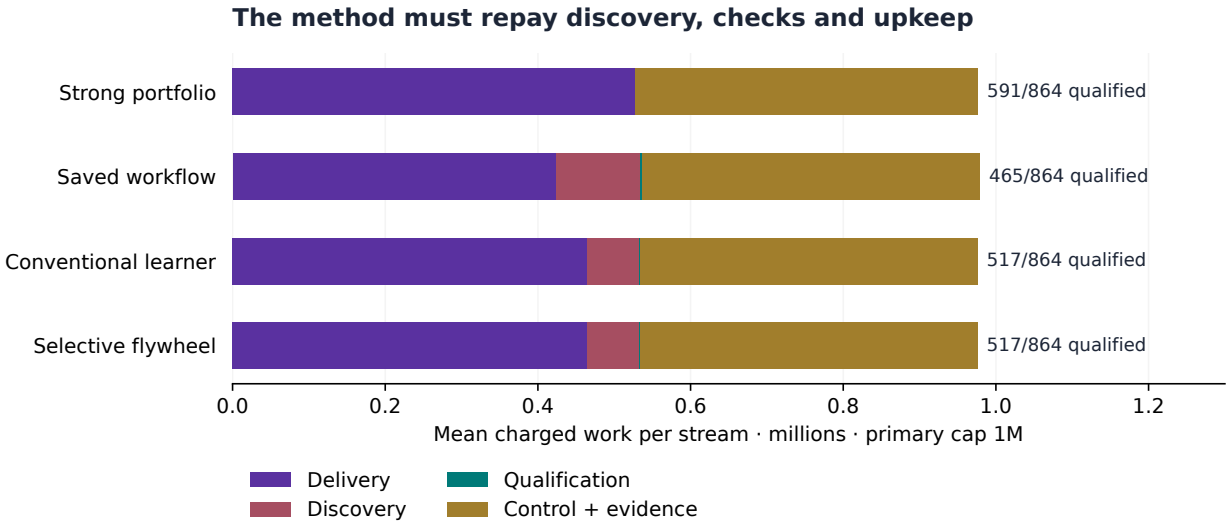


Figure 4: Primary mean charged work, including delivery, discovery, qualification, control and evidence. The strongest portfolio's larger qualified output prevents a cheaper incomplete search from being called a productive inheritance advantage.

repairs ignore that parent at delivery, and the selective repair takes its stable branch. Parent references alone therefore cannot support a claim of useful deep succession. The four mission positions across 12 streams yield 48 valid plans and 36 deadline-qualified deliveries: 12/12 founders and 24/36 later missions. Every one of the 36 retained assets is labeled `DIAGNOSTIC_CORRECTNESS_ONLY`; none passes the registered full-investment forecast in that inherited treatment.

Ancestry treatment	Valid / 48	Qualified / 48	Mean work	Lineages / 12
Inherited	48	36	888,009.5	12
Removed	48	**37**	**225,339.3**	0
Rebuilt	48	36	2,239,765.9	12
Equivalent replacement	48	36	889,359.1	12

Table 6: Separate 5,000,000-unit diagnostic ceiling; all four missions include the founder. Rebuilding and equivalent replacement preserve every fresh schedule in all 12 paired streams. Removing ancestry prevents the authored child construction while leaving the conventional planner available.

Rebuilding recovers the exact ancestral content on all 36 reconstruction events. Inheritance uses **60.35% less work** than rebuilding. Equivalent normalized replacements preserve every fresh schedule, supporting equivalence of the substituted method behavior rather than dependence on a label. These interventions demonstrate conditional reconstruction savings; they do not by themselves prove that every retained child needs its recorded parent. The strongest executed succession claim is dispatch-to-lookahead on fresh work, plus parent use in repair construction/qualification. Parent-call counters are producer instrumentation; the independent audit attributes and reconciles them against exact raw records, while the separate checker verifies returned outputs. A call alone proves neither necessity nor value.

They do not support a net delivery advantage. Removing ancestry leaves no three-stage chain, yet the strong conventional fallback qualifies one more mission and costs substantially less. Eight of 12 removed-parent streams change at least one schedule. This preserves negative transfer and a cheaper alternative. In one descriptive fresh lookahead case, stream 10, inheritance reduces makespan from 50 to 47 while both versions qualify, at 16,214 versus 9,704 mission units; the complete 12-pair record also contains the losses. This isolated quality gain does not establish full-investment payback or survival against every specialist. The diagnostic demonstrates what a retained method can construct; it does not show that constructing it is the best way to satisfy the current finite demand.

An inspectable first lineage. The first stream supplies a concrete four-mission trace. The founder plan completes at slot 47. A learned critical-path rule then serves a different ordinary task at slot 39, meeting its independently checked lower bound. Its width-2, one-step lookahead child serves a tighter task at slot 36

but misses its deadline. A quality-oriented repair child preserves locked work and finishes at slot 51, with displacement 14, also missing its deadline. The latter misses remain in the record. The parent/child digests begin `7cebe063`, `7040e32c` and `0b17530e`; full ASTs, scopes, checks and lineage are in the first diagnostic checkpoint. This stream costs 869,191 units overall.

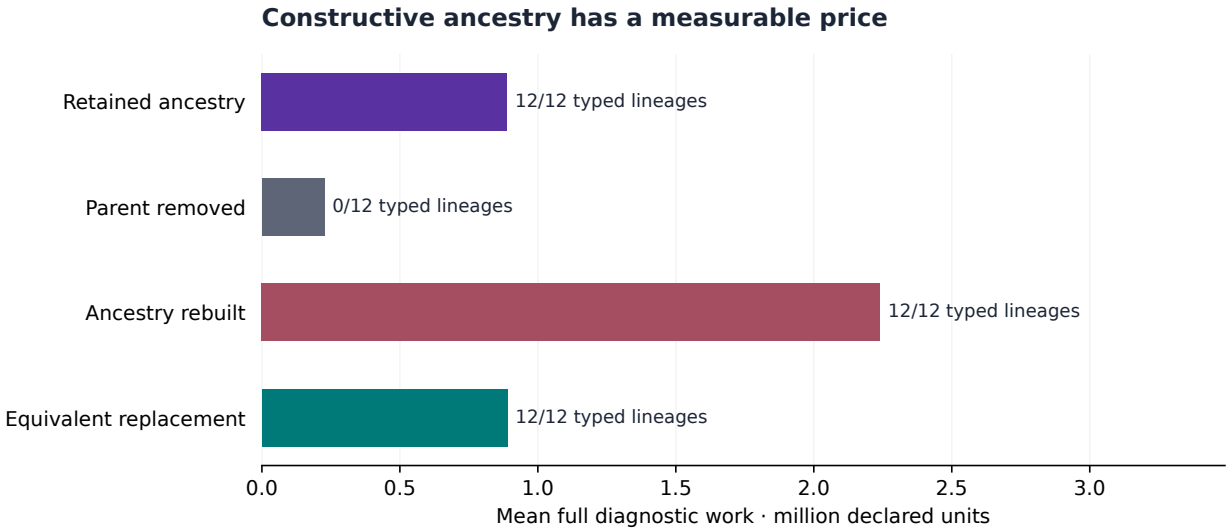


Figure 5: The four ancestry interventions have distinct meanings. Removing a parent may block its child while leaving an effective conventional solver available. Rebuilding pays acquisition again; a semantically equivalent replacement tests dependence on useful behavior rather than the asset's label. Correctness-only diagnostic retention is separate from primary economic admission.

5.3 The native specialist remains a serious alternative

On the prespecified 36-task subset, the fixed interpreted portfolio produces 36 valid plans and meets 27 deadlines, at a measured mean preparation/check interval of 20.13 ms. The native deadline model meets 31 deadlines. The solver reports four deadline models infeasible and reaches one solve limit without a qualifying result. Its mean native-and-check interval is 278.11 ms; the practical workflow that also retains the known portfolio fallback averages 298.24 ms and preserves 36 valid results.

The makespan model likewise qualifies 31/36 and returns 36 valid plans. It reports 29 optimal objective statuses and seven time-limited incumbents; the raw gap and dual-bound fields are preserved. Its mean end-to-end interval, including the independently checked portfolio upper bound, is 1,021.78 ms. Shared native process startup adds 405.02 ms to the campaign. These are measured physical intervals on one host, not conversions from the primary work tariff. A deadline mode's optimal status concerns its zero-objective feasibility model, not optimal makespan; returned feasible schedules are independently checked, while infeasibility and optimality statuses are solver reports without separately verified proof certificates.

The additional four qualified deadlines come from a strong established specialist. They are not improvements discovered by the flywheel. The reference illustrates a practical quality/latency choice that a future hybrid may exploit under a common, prospectively measured budget. The present experiment does not establish such a hybrid's overall superiority or price its extra computation as revenue.

5.4 Counterexamples improve screening before they repay acquisition

The separately frozen development probe executes the DOCX's challenge process. Three seeded defective creator artifacts produce substantiated objections, corrected outputs and retained regression cases. Two objections come from the bounded challenger; the locked-bound case is a review-specified counterexample. A fourth, deliberately false objection claims that overlapping jobs necessarily violate a pooled crew constraint. The adjudicator refutes it because concurrent demand equals available capacity. The correct result is retained.

The probe then compares seven prespecified candidate variants on 14 new diagnostic cases, with no protected confirmation inputs. Without early assets, the unchanged final checker uses 70 calls and 105,994 reference units. Screening the retained counterexamples first reduces these to 37 calls and 53,721 units. Acquisition

and retention cost 41,785 units, while retrieval adds 27,177. The first batch therefore costs **122,683 units**, a **16,689-unit loss**. The conventional regression suite given the same assets produces exactly the same checks, costs and decisions.

All arms retain one valid candidate and reject six flawed variants. Final correctness does not improve. The useful change is *earlier* exposure of particular defects; the valid candidate pays additional screening overhead. Repeating the identical batch would save 25,096 recurring units after retrieval, giving a modeled two-batch acquisition payback. That calculation assumes the same future defect mix and does not establish observed demand, a population effect or profit.

This probe's frozen tariff charges one reference unit per retained or retrieved byte, unlike the main study's 64-byte envelope/checkpoint blocks. Its payback boundary is conditional on that declared model. The two unit systems are not added together or treated as physical CPU costs.

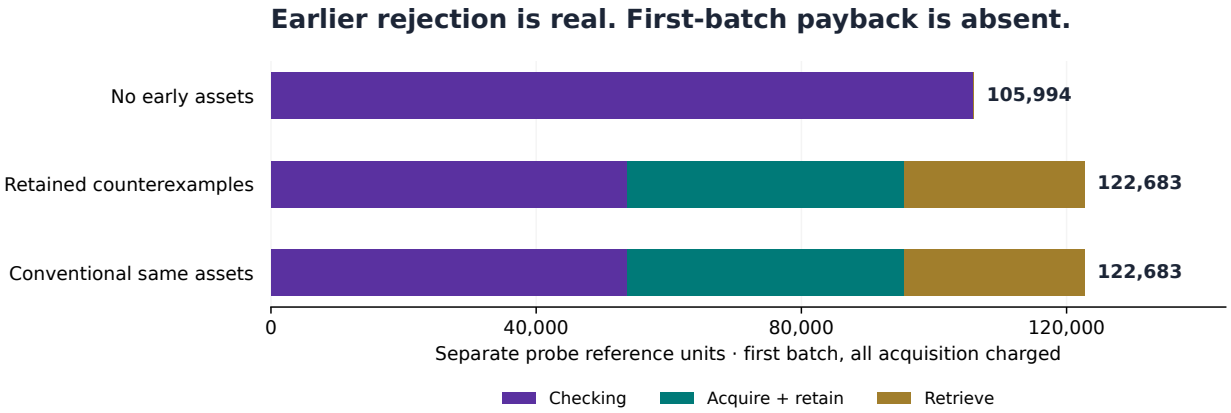


Figure 6: A verified counterexample is an asset with a carrying cost. Earlier rejection cuts checking work, but acquisition and retrieval reverse the first-batch saving. The conventional asset control is exactly equal. These are the separate development probe's reference units, not pooled main-study work units.

The raw challenge evidence, independent adjudications, revisions, regression assets, prospective probe protocol and frozen source identities are included under `review/`. The probe's seven deliberately seeded variants are not a random sample of production defects; no statistical population claim is made. Evaluator source and the final acceptance predicate remain unchanged throughout.

6 A recurring loop must survive its own operating history

6.1 One start, durable boundaries and selective stopping

The research launcher explicitly starts one finite campaign. Within each stream the released workbench adapter produces the result pack while the research controller advances Identify, discovery, construction, qualification, admission and the next permitted mission. It persists accepted results, rejected candidates, charged work and source-bound methods. The end of useful authorized work is a WAIT/COMPLETE outcome, not an instruction to manufacture more demand.

A separate-process fault campaign executed 12 checks before confirmation. Its source identity is preserved as `e395004b`; the final `7e5a1dd2` bundle adds native fallback reporting, and the review records that exact delta instead of relabelling earlier tests. The exercised operating mechanisms and separate checker remain unchanged. All 12 checks pass in that campaign, with raw failed-process exit codes, checkpoints, guards and result receipts retained.

A durable intent reserves the next execution. A ready result permits recovery without recomputing an already accepted effect. A monotone guard retains expenditure, sequence and rejection history so that a well-formed but stale snapshot cannot refund paid discovery. Recovery still performs a new check: integrity and freshness do not establish correctness by themselves. The local guard and hashes do not defend against an administrator who can rewrite the entire host and all evidence.

Operating condition	Executed observation
Restart / lost acknowledgement	A process exits after durable acceptance; restart completes three missions with three mission executions. Repeating the completed request adds no execution.
Interrupted intent	An unacknowledged attempt charges its full 94,284-unit reservation before recovery. Exactly-once CPU execution is not claimed.
STOP and resume	The first accepted receipt remains byte-identified; explicit resume completes the finite stream.
Changed dependency or source	The old snapshot is refused against the current configuration/source identity.
Invalid receipt	A changed schedule with recomputed outer hashes still fails the separate current checker.
Budget, expiry and wall deadline	Exhausted work, expired or malformed authority and a finite wall boundary stop further work. No retry renews the limit.
Stale state / rejected descendant	An older same-cursor paid checkpoint is refused; rollback restores the portfolio; retry or rehashed restore cannot resurrect the rejected asset.
Unresponsive checker	The subprocess times out, delivers no accepted result and charges the 3,620-unit checker reservation.

Table 7: Twelve source-bound development checks grouped by failure boundary. Raw evidence is under `developmen t/operating-faults-final/`; the exact-source review is `review/MAIN_PRE_FREEZE_REVIEW.json`.

6.2 Storage, maintenance and bounded load

The controller limits the active catalogue to three source-bound assets. Lookup, retrieval, current checking, admission and checkpoint growth are paid by the declared tariff. Replaced, stale or rolled-back assets lose applicability; their evidence remains in the research archive. Keeping historical failures is useful for audit but is not a zero-cost production retention policy. The finite experiment reports physical stored bytes and charges serialization blocks. It does not establish indefinite archival economics or a storage-leak-free long-running service.

The registered load probe completes 85/85 valid schedules, of which 68 meet their deadlines, in batches of 1, 4, 16 and 64 with at most four concurrent checker processes. Observed throughput rises from 21.06 to 37.07, 51.09 and 56.95 tasks per second; the 95th-percentile enqueue-to-durable latency rises from 37.63 to 89.63, 288.45 and 1,018.46 ms. No batch exhausts its deadline, and no mid-batch manual intervention occurs. These are finite local measurements, including queueing, input preparation, evidence writes and checker startup/shutdown. They do not establish sustained throughput under arbitrary load.

The complete confirmatory campaign uses one explicit launch for 336 stream/treatment executions and 72 native reference calls, with zero manually triggered intermediate phases. It takes 844.40 seconds of elapsed time and 751.10 seconds of measured Node CPU; the separate Python CPU total is unavailable. Common synthetic WORLD construction takes 420.61 ms and includes 3,464,226 known baseline-generation units, reported separately from conditional per-arm mission work. Setup, authoring, review and the additional historical replay remain research effort, not free autonomous output.

The 336 completed trial directories occupy 209,075,637 bytes in total, with final checkpoints ranging from 89,913 to 442,729 bytes. In the primary flywheel arm, checkpoint serialization alone costs 5,238,180 of 11,714,786 charged units (44.71%). Retention cannot be evaluated as though its history were free. These full research snapshots are not an optimal archival design; an incrementally stored journal could change this cost boundary and would require its own measured comparison.

6.3 The browser is an instrument, not an animation

The standalone browser opens with a nine-job synthetic release, three available workers and a 28-hour target. Its fresh computation yields a valid 23-hour plan; all nine jobs pass the separate browser checker. Global optimality is not claimed. The user can edit jobs, dependencies, durations, crew and calendar, inspect the schedule as a table or chart, and download JSON/CSV evidence. No account, installation, wallet, API key or network connection is required.

A second action starts the six-mission bounded stream. The interface shows the current result, exact retained strategy, candidate/incumbent checks, full browser charges and the next requirement. A separate construction-laboratory action labels correctness-only retention. The research-results panel is explicitly

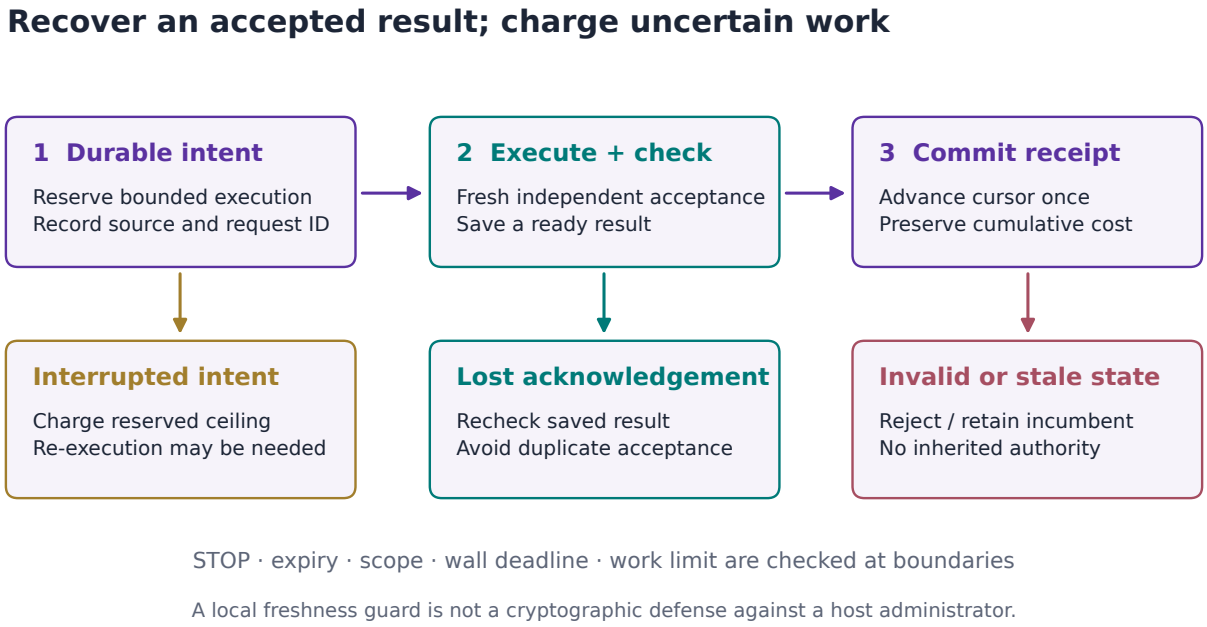


Figure 7: Failure and recovery boundaries. Accepted-result deduplication and conservative charging of uncertain work are different guarantees.

recorded evidence; opening it does not rerun confirmation. Browser checker units and the main Python-backed reference counts are distinct measurements.

One development recovery failure is preserved rather than erased: reopening identical HTML regenerated a fixture-accounting value from 6,096 to 6,115, which correctly triggered a configuration mismatch. The scientific interpreter now uses explicitly metered deterministic insertion sorting; browser fixtures have fixed source bytes and construction provenance. Final frozen-source browser checks cover 18 checker assertions, 11 useful-plan/authority/layout checks, eight stream/recovery checks and five recorded-table checks. The exact HTML hashes and the failed original state are retained.

Closing the tab stops browser computation. Saved history supports an explicit later resume. The local Node companion can continue while its explicitly started process remains running; no hosted agent, native installed operator or unattended production service is represented by the page.

7 Proof, authority and applicable $AGIALPHA utility

7.1 Four objects that must remain separate

WORLD is the current mission: jobs, durations, capacity, releases, locks and the original evidence. **POLICY** is the executable dispatch, lookahead or repair strategy. **PROOF** is a current check against the frozen contract, with the checker identity and its independence limits. **AUTHORITY** is permission to perform a particular action now, under its scope, expiry and budget. An old observation can inform a new policy without validating the next result or authorizing its execution.

The unchanged released workbench returns research packs whose production admission is HOLD and whose independent-proof flag and authority grants remain unset. The new companion's independently implemented local check is recorded as research evidence alongside that pack; it does not rewrite the upstream production verdict. Import inspection yields historical evidence with empty active proof and authority. A browser history or copied archive therefore cannot promote itself into an operating grant [21].

The research retained/incumbent decision throughout this paper concerns the finite experiment only. It is not a Harness release approval, native runtime admission or production authorization. The installed runtime, its signed journal, cycle CLI and preserved research scheduler were inspected separately. Native initialization would create local role credentials and was not performed. An unchanged source extension executed here is evidence for that extension, not for an installed four-product service.

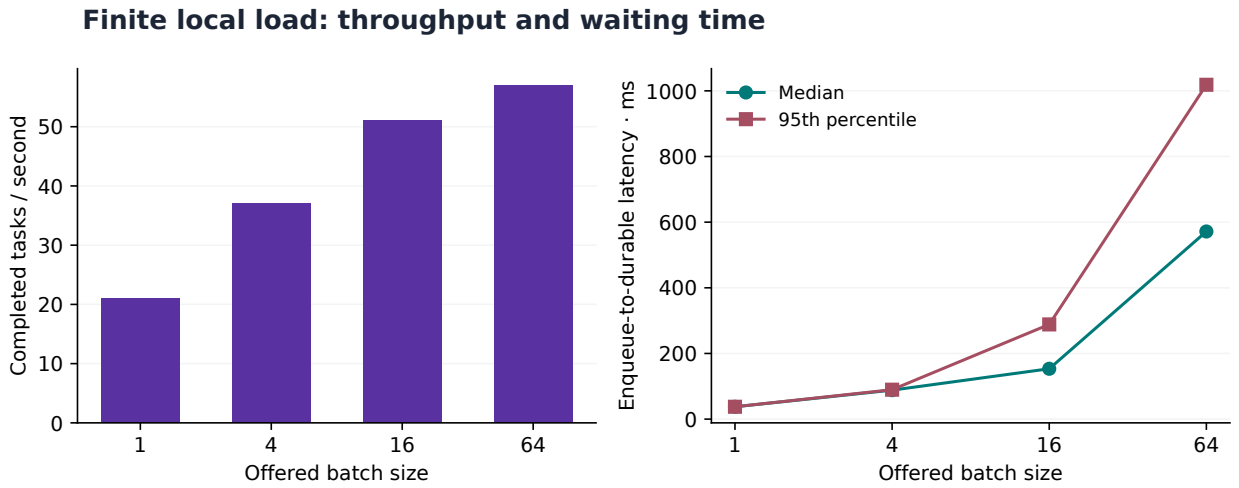


Figure 8: One local observation per batch, at most four in flight. Latency includes queueing through durable evidence write. Task mixtures differ slightly across the small batches, and the host is not a dedicated production environment. These measurements do not establish a service-level objective or general scalability.

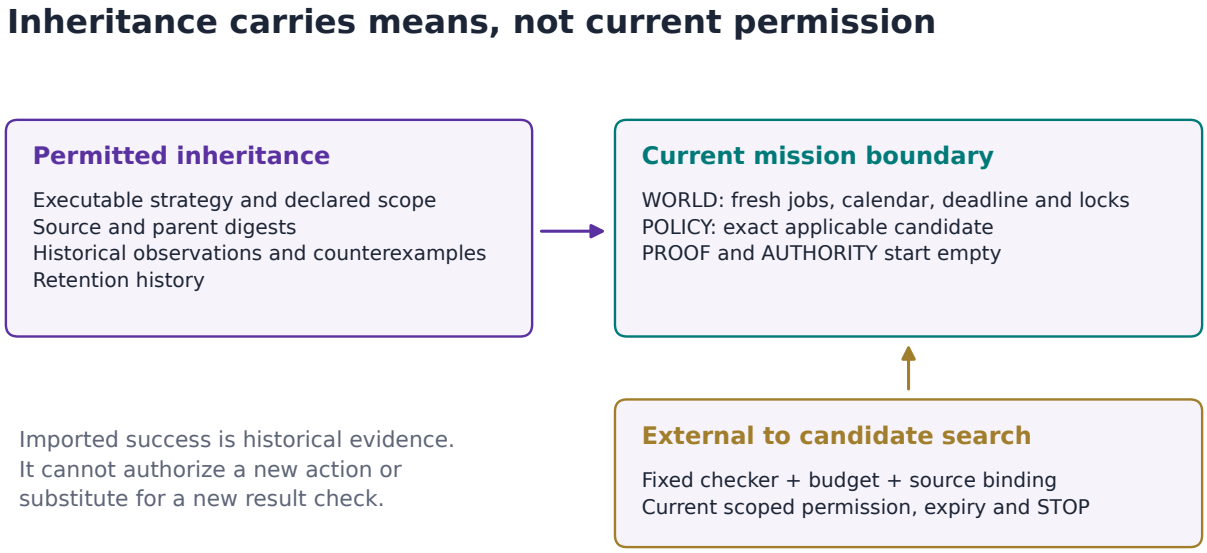


Figure 9: An artifact can cross generations with provenance. Current proof and action permission must be established separately.

7.2 Verified repository interfaces

The source snapshot covers Alpha, Jobs, Manager and Fleet at the exact revisions listed in the evidence docket. Alpha's released v1.42.0 workbench provides the executed adapter seam and unchanged AGI Jobs draft compiler. Manager and Jobs expose `discoveryLabReceiverCapabilities`, `inspectDiscoveryLabWorkRequest` and `receiveDiscoveryLabWorkRequest`. Their inspected adapters report local inspection, no provisioned external submission and no production authority. Manager names a `USDC6` route; Jobs names an `AGIALPHA18` route. They are not a shared balance or an implicit currency conversion.

Fleet's inspected `SuccessorController` exposes prepare, execute, stop, result and health operations with principal, scope, expiry, runtime/policy binding and attempt limits. This study did not run that controller. Existing source-review and release restrictions remain in force. The paper supplies no evidence that those restrictions have been cleared.

$AGIALPHA has a potential practical role when a separately commissioned work contract needs funded escrow, worker/validator commitments and attributable settlement. A sealed hash does not establish the underlying result's truth. Contract voting is not a scientific oracle. The demonstration is unsigned, unfunded and disconnected: no simulated contract execution, local EVM run, installed adapter submission or live

Alpha contract	Source-verified relevant functions	Conditional practical purpose
AscensionAccess	`register`, `owns`, `setAdmission`, `admitted`, `eligible`	Bind live ENS role ownership to a separate expiring admission.
NovaSeed	`seal`, `plan`, `creatorOf`	Commit a capsule hash, job-plan root, evidence hash and optional ancestry.
AscensionRiskOracle	`deposit/withdraw`, `lockForJob`, `openReview`, `attest`, `green`	AGIALPHA-denominated validator stake/capacity and result-bound review rounds.
AscensionJobMarket	`post/postFromSeed`, `bid`, `award`, `submit`, `validate`, `timeout`, `claimRefund`	Bounded work escrow, worker bonds, selected review, settlement or refund.
AscensionMark	`open`, `quote`, `buy/sell`, `transferLots`, `bloom`, `route`, `collect`, `close`, `reclaim`	Admitted funding lots and treasury-bounded routing of committed jobs.

Table 8: Actual source interfaces at released Alpha commit `c352bad0`. Full paths, source copies and hashes are included [22]. This table establishes source inspection only.

settlement is claimed. Token issuance, reserve movements, scarcity and price appreciation are outside the productivity endpoint.

7.3 Publication is another exact-source boundary

Issue #4850 is the existing coordination surface [23]. This work uses an isolated, additive research namespace and preserves all earlier editions. The receiving publication coordinator owns shared catalogue and deployment changes. Passive PDF publication and new interactive-source qualification are different gates. The final package provides exact artifact identities for that handoff; public links are reported only after retrieval and served-byte verification. No merge or deployment status is inferred from the existence of a local browser file.

8 What would make the flywheel accelerate?

The decisive unit is a method with a demonstrated consequence. A growing task queue, memory store or population of agents supplies no evidence of productive growth by itself. The useful test is whether a retained procedure changes the cost or feasibility of subsequent qualified work after paying for its discovery and upkeep. The same rule applies to a strategy for finding strategies: its recursive position does not exempt it from a current comparison.

8.1 From an operational loop to a growth claim

An operational loop can run correctly while making an unprofitable investment. Useful reuse can occur without expanding the feasible mission set. A parent can help construct a distinct child while a direct specialist remains cheaper. Amortization can produce a declining average cost without increasing the rate of improvement. These distinctions prevent one visible success from standing in for all the claims in the word *flywheel*.

Discovery and delivery share one cumulative account, with bounded search and selective admission. It does not implement an endogenous economy that earns resources, reinvests a fraction of realized surplus and demonstrates a causal increase in its improvement rate. Consequently, **self-acceleration is not established**. The matched mission windows describe productivity over this finite run; they cannot establish indefinite growth. A further experiment would need a preregistered resource-return mechanism, equivalent conventional learning and no-reinvestment controls, and enough independent episodes to distinguish accelerating capability from initial acquisition payback. The gate must also permit the best decision to be no further construction.

8.2 Why the first mission matters

The browser begins with an ordinary deliverable: a schedule that respects stated dependencies, pooled workers and calendar limits. Users can inspect the proposed start and finish times, edit assumptions, and export a machine-readable plan. A target miss is displayed as a target miss. A feasible plan is not automatically an optimal one, and a verified model does not certify that its duration estimates describe a real project. This is a usable computational planning instrument, not physical execution of the project.

Later work earns a reason to return only if it improves an outcome the user values. Lower checking burden, a better response to disrupted capacity or an additional feasible deadline can matter; a larger strategy archive need not. Conditional break-even calculations help decide whether to investigate further, but real willingness to pay, accepted delivery and commercial economics require separate observations. No paid customer, token price, revenue, liquidity or profit is inferred from this study.

8.3 Limits of independence, scope and cost

The producer, independent checker and adversarial review were developed as separate components within the same commissioned project. The Python checker was authored from the input/output contract without importing the producer's solver. The browser has its own implementation of the acceptance checks. This separation catches specific implementation errors; it is not independent institutional replication, formal verification or a defense against a compromised host administrator. Shared specifications can contain correlated mistakes. The recovered historical artifact audit and the newly completed exact frozen-policy replay are distinct checks, both preserved; neither establishes generalization outside the original benchmark.

The experiment samples a small synthetic scheduling family within an authored strategy language. Its fixtures do not represent the distribution of all release projects. Calendar size, job count, worker demand, grammar and deadlines are bounded; out-of-scope inputs should be refused. A finite stream may not repay an asset that would be useful under a different demand distribution. Conversely, extrapolating break-even without measuring maintenance, stale assumptions and future qualification rates can turn a promising asset into a liability.

The browser-first strategy language contains competent interpreted specialists, while the native mixed-integer optimizer is an additional strong reference. It is not a freely interchangeable action inside the counted-work policy: its opaque native work has a separately bounded timing model. This limits the experiment's claim. A practical hybrid that chooses between fast heuristics and native optimization under a common measured time-and-quality budget could be the better service. A winning specialist should be adopted on that evidence; an agentic label supplies no reason to exclude it. This edition does not claim that its controller is already the best such hybrid.

Declared work units are explicit reference operations, not energy, money, total CPU instructions or equal elapsed time. Native optimization uses a different execution model and is compared with separately disclosed wall and CPU measurements. Setup, research design, authoring, debugging and review are documented in an intervention ledger, but their human durations and model charges are unavailable. Those unknowns prevent an all-in monetary productivity claim. The bounded automatic run has its own exact interventions and resource ledger; it does not erase the work required to build the experiment.

The standard for the next successor

Retain a method because it changes fresh, verified work for the better. Retain the incumbent when it does not. Let the same test decide whether searching for the next method is worth doing.

9 Reproduce, inspect and extend

The complete companion preserves the authoritative author PDF with its LaTeX, bibliography and editable PDF/SVG figures. It also contains the frozen source and protocol, mission inputs, exact outputs, failed proposals, intervention records, independently implemented checker, historical audit and the original Strategy Laboratory archive. The preferred Edition 02 is a separate preserved publication; no file in that edition is replaced.

9.1 Three entry points

Use the planner. Open the standalone `AGI_ALPHA_Strategy_Laboratory.html` in a current browser. The default action computes a useful plan from editable synthetic inputs, checks it locally and offers JSON/CSV exports. There is no account, installation, wallet, model endpoint or API key. Browser work runs only while its page is open. A saved checkpoint permits an explicit later resume; it is not a background service.

Inspect the experiment. The evidence view clearly marks recorded confirmation. The complete archive's `EVIDENCE_DOCKET.json` connects material claims to files and source identities. `SHA256SUMS.txt` checks the packaged payload, while `protocol/STUDY_FREEZE.json` binds prospective execution and evaluation files. The development and historical directories retain their distinct evidence classes. They cannot be relabelled as fresh confirmation.

Run the companion. The root reproduction guide gives the exact clean-output command, environment pins and expected bounded outputs. Node 24.22.1 runs the workbench and strategy controller; Python 3.12.14 runs the separate checker. The native reference uses NumPy 2.4.6 and SciPy 1.17.0. The browser itself has no external library dependency. Reproduction executes into a new directory and first verifies the frozen source. It never overwrites the original recorded run or silently rebinds a changed source.

9.2 What is fixed and what is editable

The strategy language, checker, source-binding controls, discovery opportunities, task distribution, comparator policies, resource limits, primary analysis and stopping rules were committed before confirmation. A reader may study or extend them, but a change requires a new source identity and new evaluation. Reusing an old favorable outcome is not qualification of that change. The manifest is an integrity check, not a signature from an independent institution. The source identity begins `7e5a1dd2dcf7`; the 52-file prospective freeze begins `6d0308707f39`. Both full hashes and the pre-execution public record on issue #4850, comment 6390926576 are in the evidence docket. The frozen archive was saved and read back with an identical hash before the single confirmation launch.

The paper can be rebuilt with `bash paper/build.sh` using XeLaTeX and BibTeX from TeX Live 2023. Figure generation uses Matplotlib 3.10.8. The build pins metadata time, document identity and fonts. The delivered verification record reports a clean rebuild from the packaged source, page rendering and inspection, and saved-copy hash checks. The original author PDF stays inside the archive even if a document-serving system adds provenance metadata to a separately served copy.

9.3 An additive handoff

Publication artifacts use the distinct strategy-succession name. The repository handoff records the exact paper, companion, protocol, evidence and browser source identities for the existing owner on issue #4850. Source availability, a draft pull request, accepted release qualification and verified public serving are different states. A public URL is not inferred from a planned destination. All earlier experiments, installed-runtime identities and held Harness/Fleet restrictions retain their original scope.

Make every mission leave useful means behind

The strongest durable output is not an expanding history of attempts. It is a permitted method, accompanied by the evidence that says when to use it, when to repair it, and when to stop paying for it.

References

[1] Vincent Boucher. AGI ALPHA: A scalable substrate for intelligence organizations. MONTREAL.AI research manuscript, April 2026. URL https://github.com/MontrealAI/agialpha-first-real-loop/blob/main/docs/manuscript/AGI_ALPHA_Unified_Publication_Final.pdf. Architectural foundation; text recovered from the verified Alpha release mirror. Accessed 24 October 2026.

[2] Vincent Boucher. Transformational intelligence: Learning to change the game. MONTREAL.AI research manuscript, 2026. URL https://montrealai.github.io/AGI-Alpha-Agent-v0/Transformational_Intelligence.pdf. Purpose, formulation and evaluator separation. Source snapshot retained. Accessed 24 October 2026.

[3] Vincent Boucher. Out-Learn. MONTREAL.AI revised research manuscript, 2026. URL https://montrealai.github.io/AGI-Alpha-Agent-v0/Out_Learn.pdf. Current revised manuscript; experiment construction and calibration. Accessed 24 October 2026.

[4] Vincent Boucher. Out-Think. MONTREAL.AI revised research manuscript, 2026. URL https://montrealai.github.io/AGI-Alpha-Agent-v0/Out_Think.pdf. Current revised manuscript; checked representations and retirement. Accessed 24 October 2026.

[5] Vincent Boucher. Out-Design. MONTREAL.AI revised research manuscript, 2026. URL https://montrealai.github.io/AGI-Alpha-Agent-v0/Out_Design.pdf. Current revised manuscript; executable release certificates. Accessed 24 October 2026.

[6] Vincent Boucher. Out-Strategise. MONTREAL.AI revised research manuscript, 2026. URL https://montrealai.github.io/AGI-Alpha-Agent-v0/Out_Strategise.pdf. Current revised manuscript; observation and common-completion construction. Accessed 24 October 2026.

[7] Vincent Boucher. Out-Execute. MONTREAL.AI revised research manuscript, 2026. URL https://montrealai.github.io/AGI-Alpha-Agent-v0/Out_Execute.pdf. Current revised manuscript; reusable boundary contracts. Accessed 24 October 2026.

[8] Vincent Boucher. Out-Evolve: Building the means of discovery and production. MONTREAL.AI revised research manuscript, October 2026. URL https://montrealai.github.io/AGI-Alpha-Agent-v0/Out_Evolve.pdf. 20 October revision, 29 pages. Public PDF recovered and hashed. Accessed 24 October 2026.

[9] SUCCESSOR Ω — Strategy Laboratory v0.1.0. Recovered original research archive; included with the research companion, 2026. Original archive, protocols, raw evidence and logs included. Recorded exact runtime: CPython 3.13.5.

[10] 202610180921: Strategy and evidence-grounded challenge. User-supplied conversational research lead, DOCX, 2026. Complete original DOCX included. Methodological lead; not independent experimental evidence.

[11] MONTREAL.AI. The possibility foundry: Autonomous enterprises that invent their next capability. Research edition 01, October 2026. URL https://montrealai.github.io/AGI-Alpha-Agent-v0/papers/possibility-foundry-v1.pdf. Research direction: Vincent Boucher. Source-reported synthetic instrument and successor results. Accessed 24 October 2026.

[12] MONTREAL.AI. SUCCESSOR Ω harness: Make every mission build what comes next. Experimental research edition, October 2026. URL https://montrealai.github.io/AGI-Alpha-Agent-v0/papers/successor-omega-harness-detector-v1.pdf. 24 October 2026, 28 pages. Synthetic parent-to-successor detector experiment.

[13] MONTREAL.AI. Neural-symbolic SUCCESSOR Ω: Customer-owned mission intelligence. Definitive corporate white paper, version 1, August 2026. URL https://montrealai.github.io/neuralsymbolicsuccessoromegav1.pdf. 25 August 2026, 59 pages. Architecture and product constitution. Public PDF recovered and hashed.

[14] Vincent Boucher. Gym, specialist ASI and successor Ω: The sovereign mission environment, the proven intelligence institution, and the successor to the successor in singularity time. MONTREAL.AI research, version 2.1.0, August 2026. URL https://montrealai.github.io/GoalOS_Gym_Specialist_ASI_Successor_Omega_v2_1_0_US_LETTER.pdf. 10 August 2026, 83 PDF pages. Public PDF recovered and hashed.

[15] Vincent Boucher. SUCCESSOR Ω × AGI JOBS: The constitutional compiler and proof-carrying work system for mission-sovereign intelligence institutions. MONTREAL.AI research, version 2.0.0, August 2026. URL https://montrealai.github.io/successor-omega-x-agi-jobs-v2.pdf. 12 August 2026, 110 PDF pages. Public PDF recovered and hashed.

[16] MONTREAL.AI. AGI ALPHA FLYWHEEL: From completed work to compounding capability. Preferred published Edition 02, October 2026. URL https://montrealai.github.io/AGI-Alpha-Agent-v0/papers/agi-alpha-flywheel-acceleration-v2.pdf. 18 pages. Preferred published edition preserved; served bytes verified 24 October 2026.

[17] Rainer Kolisch. Serial and parallel resource-constrained project scheduling methods revisited: Theory and computation. *European Journal of Operational Research*, 90(2):320–333, 1996. doi: 10.1016/0377-2217(95)00357-6. URL https://doi.org/10.1016/0377-2217(95)00357-6.

[18] Dimitri P. Bertsekas, John N. Tsitsiklis, and Cynara Wu. Rollout algorithms for combinatorial optimization. *Journal of Heuristics*, 3:245–262, 1997. URL https://www.mit.edu/~jnt/Papers/J066-97-rollout.pdf.

[19] SciPy developers. scipy.optimize.milp. SciPy 1.17.0 reference documentation, 2026. URL https://docs.scipy.org/doc/scipy-1.17.0/reference/generated/scipy.optimize.milp.html. Version matched to the executed research environment; accessed 24 October 2026.

[20] HiGHS developers. HiGHS: High-performance software for linear optimization, 2026. URL https://highs.dev/. Primary solver documentation; accessed 24 October 2026.

[21] MONTREAL.AI. AGI ALPHA v1.42.0: Successor workbench extension interface. Released software and source documentation, 2026. URL https://github.com/MontrealAI/AGI-Alpha-Agent-v0/blob/c352bad0a0941fdccd490a7224e3efd1b8de45d5/docs/agent/SUCCESSOR_WORK.md. Released source commit c352bad0a094. Exact eight-file closure and full hashes retained.

[22] MONTREAL.AI. Alpha ascension contracts: version 1.42.0 source. Five contract interfaces and shared token path, source inspected, 2026. URL https://github.com/MontrealAI/AGI-Alpha-Agent-v0/tree/c352bad0a0941fdccd490a7224e3efd1b8de45d5/contracts/ascension. Exact source copies and SHA-256 identities accompany this edition. No deployed chain state or settlement verified.

[23] MONTREAL.AI contributors. Successor operating and publication coordination, issue 4850, 2026. URL https://github.com/MontrealAI/AGI-Alpha-Agent-v0/issues/4850. Owner records retained through 24 October 2026. Source-reported statuses remain distinct from new experiments.